FP-Devicer: Open-Source Digital Fingerprinting Middleware

Technical Whitepaper — Version 1.5.5+

Gateway Corporate Solutions LLC

March 2026

Abstract

FP-Devicer is a lightweight, highly extensible TypeScript middleware library for server-side digital device fingerprinting. It computes a **confidence score (0–100)** between two fingerprint datasets by combining **weighted structural field-by-field comparison** with **TLSH (Trend Micro Locality Sensitive Hash) fuzzy holistic scoring**.

The library provides a simple `calculateConfidence` API for one-off comparisons and a full-featured `DeviceManager` paired with a pluggable `StorageAdapter` system for production-grade device identification, snapshot persistence, deduplication, adaptive weighting based on historical signal stability, and structured observability.

Key design goals:

- Near-universal server compatibility (Express, Fastify, Deno/Oak, etc.)
- Extensibility via a global plugin registry (custom weights and comparators per field path)
- High accuracy (>99% in benchmarks) with sub-millisecond per-comparison latency
- Multiple storage backends: in-memory, SQLite, PostgreSQL, and Redis

The codebase (~2,500 lines of clean TypeScript) is modular, well-tested, benchmarked, and documented via TypeDoc. It pairs naturally with the companion client-side collector **FP-Snatch** for a complete end-to-end fingerprinting solution.

Introduction & Motivation

Traditional fingerprinting libraries (e.g., FingerprintJS open-source) focus on client-side collection and hash generation. FP-Devicer shifts the intelligence to the **server side**, enabling capabilities that are impossible or impractical in the browser:

- **Persistent device IDs** that survive across browser sessions, private mode tabs, and even across different browsers on the same device
- **Adaptive scoring** that automatically down-weights signals that have proven historically volatile for a given device (e.g., screen orientation, timezone)
- **Candidate pre-filtering** to keep database queries fast even at scale — only broadly-similar fingerprints are loaded for full scoring
- **Enterprise features** such as user account linking, IP address logging, and structured metrics emission

FP-Devicer was released open-source in 2025 and has seen active development (93+ commits), with the most recent patch on March 8, 2026. It is available on GitHub at https://github.com/gatewaycorporate/fp-devicer and its generated TypeDoc reference documentation is hosted at https://gatewaycorporate.github.io/fp-devicer/.

Architecture Overview

FP-Devicer is structured into four logical layers, each with a clearly defined responsibility:

Data Model (`src/types/data.ts`) : Defines `FPUserDataSet`, the canonical interface for all fingerprint signals collected from a browser, and the generic `FPDataSet<T>` alias. Also contains configuration types such as `ComparisonOptions` and `Comparator`.

Scoring Engine (`src/libs/confidence.ts + src/libs/registry.ts`) : Implements the two-strategy hybrid confidence calculation: recursive structural scoring and TLSH fuzzy hashing. The registry is a global singleton that stores per-path weights and comparator functions contributed by built-in defaults or user plugins.

Persistence & Orchestration (`src/core/DeviceManager.ts + src/libs/adapters/`) : The `DeviceManager` class runs the full fingerprint matching pipeline. Storage adapters implement a common interface so that the engine code is entirely decoupled from the choice of database.

Observability (`src/types/observability.ts + src/libs/default-observability.ts`) : Defines injectable `Logger` and `Metrics` interfaces with no-op defaults, allowing callers to route telemetry to any logging or metrics platform.

The relationships among components are summarized in the class diagram below.

```
┌──────────────────────────────────────────────────────────────┐
│                        FP-Devicer                            │
│                                                              │
│  FPDataSet ──► ConfidenceCalculator ◄── Registry             │
│                        ▲                                     │
│                        │                                     │
│  StorageAdapter ──► DeviceManager                            │
│        ▲               │                                     │
```

```
│  (SQLite / Postgres     └──► ConfidenceCalculator            │
│   Redis / InMemory)                                          │
└──────────────────────────────────────────────────────────────┘
```

Core Components

Fingerprint Data Model

The `FPUserDataSet` interface (defined in `src/types/data.ts`) is the canonical schema for all browser signals that FP-Snatch collects client-side and FP-Devicer consumes server-side. It contains more than 30 optional fields, allowing partial fingerprints to be processed gracefully. Relevant fields include:

Field	Type	Description
`userAgent`	`string`	Full browser user-agent string
`platform`	`string`	OS platform string (e.g., `"Win32"`)
`screen`	`object`	Width, height, color depth, orientation
`fonts`	`string[]`	Enumerated installed system fonts
`canvas`	`string`	Hash of a rendered canvas element
`webgl`	`string`	Hash of WebGL rendering output
`audio`	`string`	Hash of an AudioContext fingerprint
`plugins`	`object[]`	Browser plugin names and descriptions
`mimeTypes`	`object[]`	Supported MIME types
`hardwareConcurrency`	`number`	Logical CPU core count
`deviceMemory`	`number`	Reported device RAM (GiB, rounded)
`highEntropyValues`	`object`	UA-CH high-entropy hints (brands, model, etc.)
`language` / `languages`	`string` / `string[]`	Browser locale information
`timezone`	`string`	IANA timezone identifier

All fields are optional. The scoring engine treats a field that is absent on one side of the comparison as a zero-similarity match for that path rather than throwing an error.

The generic `FPDataSet<T>` alias allows the scoring engine to operate on shapes other than `FPUserDataSet`, supporting advanced use cases where callers supply custom data models:

```
export type FPDataSet<T = FPUserDataSet> = T;
```

Confidence Scoring Engine

The scoring engine lives in `src/libs/confidence.ts` and is the intellectual core of FP-Devicer. It exposes two entry points:

- `calculateConfidence` — a pre-built default calculator, exported directly for simple use cases.
- `createConfidenceCalculator(options)` — a factory that returns a calculator instance scoped to a specific `ComparisonOptions` configuration.

Strategy 1: Weighted Structural Comparison

The structural comparison is performed by `compareRecursive`, an internal function that walks both fingerprint objects simultaneously using dot-notation paths. At each node it determines whether the value is:

- **Primitive / leaf** — delegates to the registered comparator for that path (or the built-in default) and returns a similarity in `[0, 1]`.
- **Array** — pairs elements by index and recurses into each pair, accumulating weighted contributions from each element.
- **Object** — takes the union of keys present in either object and recurses into each key pair.

Weights are retrieved via `getWeight(path)`, which consults the merged weight table built from `userOptions.weights`, the global registry, and built-in `DEFAULT_WEIGHTS`. The weights are **normalized internally**, meaning only relative magnitudes matter; it is not necessary for them to sum to any particular value.

The structural score is computed as:

$$S_{\text{structural}} = \frac{\sum_i w_i \cdot \text{sim}_i}{\sum_i w_i}$$

where w_i is the effective weight for path i and $\text{sim}_i \in [0, 1]$ is the similarity returned by the path's comparator.

Strategy 2: TLSH Fuzzy Hashing

TLSH (Trend Micro Locality Sensitive Hash) is a fuzzy hashing algorithm designed so that similar inputs produce similar hashes. Unlike cryptographic hashes, two TLSH hashes can be compared to produce a numeric distance score that correlates with the semantic similarity of the original inputs.

FP-Devicer serializes each fingerprint to a canonical JSON string (key-sorted, deterministic) before hashing. The hash distance is normalized to a `[0, 1]` similarity score:

$$S_{\text{tlsh}} = 1 - \frac{\text{distance}\ (h_1, h_2)}{\text{maxDistance}}$$

TLSH serves as a holistic cross-check. Because it operates on the serialized whole rather than individual fields, it catches fingerprint-wide shifts that the structural scorer might partially miss when individual fields receive low weight.

Score Blending

The two strategies are combined by a configurable `tlshWeight` parameter (default `0.30`):

$$S_{\text{final}} = S_{\text{structural}} \cdot (1 - w_{\text{tlsh}}) + S_{\text{tlsh}} \cdot w_{\text{tlsh}}$$

The final result is scaled to the integer range `[0, 100]` and returned. A score of `100` indicates an exact or near-exact match; a score of `0` indicates no detectable similarity.

Options Resolution Order

When `createConfidenceCalculator` builds its internal weight and comparator tables it merges sources in the following priority order (highest first):

1. `userOptions.weights` / `userOptions.comparators` (caller-supplied overrides)
2. Built-in `DEFAULT_WEIGHTS` (hardcoded high-entropy emphasis)
3. Global registry entries added via `registerPlugin` / `registerWeight`
4. `defaultWeight` fallback (default `1`)

ComparisonOptions Reference

```typescript
interface ComparisonOptions {
    /** Per-path weights. Normalized automatically. */
    weights?: Record<string, number>;
    /** Per-path custom similarity functions. */
    comparators?: Record<string, Comparator>;
    /** Fallback weight for paths without an explicit entry. Default: 1 */
    defaultWeight?: number;
    /** TLSH blend factor in [0, 1]. Default: 0.30 */
    tlshWeight?: number;
    /** Maximum object recursion depth. Default: 8 */
    maxDepth?: number;
    /** Whether to consult the global registry. Default: true */
    useGlobalRegistry?: boolean;
}
```

Plugin & Registry System

The registry (`src/libs/registry.ts`) is a **global singleton** that stores per-path comparators and weights. It is lazily seeded by `initializeDefaultRegistry()` (called from `src/libs/default-plugins.ts`) the first time any scoring function runs.

Built-in defaults include:

- **Jaccard similarity** for set-like arrays (`fonts`, `languages`, `plugins`, `mimeTypes`) — measures the ratio of the intersection size to the union size.
- **Exact match** for hash fields (`canvas`, `webgl`, `audio`) — returns `1.0` only when both values are identical strings.

- **`screenSimilarity`** for the `screen` object — a custom function that tolerates small pixel-rounding differences while penalizing large resolution changes.
- **Structural equality** as a fallback for primitives with no registered comparator.

Callers can extend or override the registry at any time using the exported helper functions:

```
// Register both a custom weight and a custom comparator at once
registerPlugin("userAgent", {
    weight: 25,
    comparator: (a, b) =>
        levenshteinSimilarity(
            String(a ?? "").toLowerCase(),
            String(b ?? "").toLowerCase(),
        ),
});

// Register only a weight (keep the existing comparator)
registerWeight("canvas", 40);

// Register only a comparator (keep the existing weight)
registerComparator("timezone", (a, b) => (a === b ? 1 : 0.5));

// Set the fallback weight for any unregistered path
setDefaultWeight(2);

// Remove a registration
unregisterWeight("audio");
unregisterComparator("audio");
```

Registry changes take effect immediately for all subsequent calls to `calculateConfidence` (including via `DeviceManager`) because the global registry is consulted at scoring time, not at calculator-creation time.

Device Management

`DeviceManager` (`src/core/DeviceManager.ts`) is the high-level orchestrator that makes FP-Devicer suitable for production use. It wraps the scoring engine with a complete fingerprint matching pipeline:

Pipeline Steps

1. **Deduplication cache** — Computes the TLSH hash of the incoming fingerprint and checks an in-memory LRU cache (keyed by hash, 5-second TTL). If the same fingerprint is seen twice within the window, the cached `IdentifyResult` is returned immediately without any database I/O. This is critical for endpoints that receive burst traffic.

2. **Candidate pre-filtering** — Calls `adapter.findCandidates(incoming, minScore)` to retrieve a small set of stored device snapshots that are broadly similar to the incoming data. Each adapter implements this differently: the SQLite and PostgreSQL adapters

use JSON-operator `WHERE` clauses on indexed fields; the in-memory adapter does a linear scan with early termination. The default `candidateMinScore` is `30`, meaning only candidates where the adapter's rough score exceeds 30 are returned.

3. **Adaptive weighting** — For each candidate, `DeviceManager` calls `adapter.getHistory(deviceId, limit=5)` to retrieve recent snapshots and passes them to `computeFieldStabilities`. This function measures, for each fingerprint path, how consistent the field's value has been across the historical snapshots. Fields that have varied frequently (e.g., `timezone` for a travelling user) are assigned a reduced weight for this comparison, preventing transient signal drift from causing a false negative.
4. **Full confidence scoring** — Each candidate is scored against its most recent snapshot using a `createConfidenceCalculator` instance configured with the adaptive weights computed in step 3. The candidate with the highest score is selected as `bestMatch`.
5. **Decision** — If `bestMatch.confidence >= matchThreshold` (default `50`), its existing device ID is reused. Otherwise a new UUID-based device ID is minted, marking the visit as a new device.
6. **Persistence** — The incoming fingerprint is saved via `adapter.save()` as a new snapshot associated with the resolved device ID.
7. **Observability** — Structured log entries are emitted via the injected `Logger`. Metrics counters and gauges are incremented via the injected `Metrics` instance. Both default to no-op implementations.

Constructor Options

```
new DeviceManager(adapter, {
  matchThreshold?: number;      // min score to reuse a device ID (default: 50)
  candidateMinScore?: number;   // pre-filter floor passed to adapter (default:
30)
  dedupWindowMs?: number;       // dedup cache TTL in milliseconds (default:
5000)
  comparisonOptions?: ComparisonOptions; // forwarded to
createConfidenceCalculator
  observability?: {
    logger?: Logger;
    metrics?: Metrics;
  };
});
```

Return Value

```
interface IdentifyResult {
    deviceId: string; // stable UUID for this device
    confidence: number; // 0–100 score against the best matched snapshot
    isNewDevice: boolean; // true if no candidate exceeded matchThreshold
    linkedUserId?: string; // if a userId was supplied and stored
}
```

Storage Adapters

All adapters implement the `StorageAdapter` interface defined in `src/types/storage.ts`. The interface methods are:

Method	Description
`init()`	Create tables / open connections. Must be called before use.
`save(snapshot)`	Persist a new fingerprint snapshot.
`findCandidates(fp, n)`	Return up to `n` broadly-similar stored snapshots.
`getHistory(id, limit)`	Return recent snapshots for a given device ID.
`getAllFingerprints()`	Return all stored snapshots (e.g., for bulk analysis).
`linkToUser(id, uid)`	Associate a device ID with an application user ID.
`deleteOldSnapshots()`	Prune snapshots older than a configured retention window.
`close()`	(Optional) Gracefully close database connections.

The four provided adapter implementations handle the specifics:

In-memory (`createInMemoryAdapter()`) : Stores snapshots in a `Map`. Suitable for development, testing, and short-lived server processes. No persistence across restarts.

SQLite (`createSqliteAdapter(path)`) : Uses `better-sqlite3` (or the Deno-compatible equivalent). Creates a `fingerprints` table on first `init()`. The candidate pre-filter runs a `WHERE` clause that compares stored platform and screen values before loading the full rows. Ideal for single-process servers without external dependencies.

PostgreSQL (`createPostgresAdapter(connectionString)`) : Uses Drizzle ORM (`drizzle-orm/postgres-js`). Suitable for multi-process or horizontally-scaled deployments. The candidate pre-filter leverages PostgreSQL JSON operators for efficient indexed querying.

Redis (`createRedisAdapter(url)`) : Stores snapshots as JSON-serialized Redis hashes. Suitable for deployments that already operate a Redis cluster or that require very fast writes-per-second with eventual persistence.

The `AdapterFactory` class provides a configuration-driven factory for selecting and instantiating adapters without importing each adapter module directly:

```
const adapter = AdapterFactory.create("sqlite", {
    sqlite: { path: "./fingerprints.db" },
});
```

Code Paths & Sequence Diagrams

Simple Confidence Calculation

The simplest use case requires no instance management:

```
Client
  │
  ├─► calculateConfidence(fp1, fp2)
  │       │
  │       ├─► initializeDefaultRegistry()  [lazy, once only]
  │       │       │
  │       │       └─► registerPlugin × N   [canvas, fonts, screen, ...]
  │       │
  │       ├─► compareRecursive(fp1, fp2)
  │       │       │
  │       │       └─► for each field path:
  │       │               getWeight(path)
  │       │               getComparator(path)(val1, val2)
  │       │
  │       ├─► getHash(canonicalizedStringify(fp1))
  │       ├─► getHash(canonicalizedStringify(fp2))
  │       └─► compareHashes(h1, h2)  → tlsh score
  │
  └◄─ blended integer score [0–100]
```

Full Device Identification

```
HTTP Request
  │
  ├─► DeviceManager.identify(incomingFP, { userId, ip })
  │       │
  │       ├─► getHash(incoming)  →  dedupKey
  │       ├─► dedupCache.get(dedupKey)
  │       │       ├─ HIT  → return cached IdentifyResult immediately
  │       │       └─ MISS → continue
  │       │
  │       ├─► adapter.findCandidates(incoming, candidateMinScore)
  │       │       └─► returns [ candidate₁, candidate₂, ... ]
  │       │
  │       ├─► for each candidate:
  │       │       ├─► adapter.getHistory(candidate.deviceId, 5)
  │       │       ├─► computeFieldStabilities(history)
  │       │       ├─► createConfidenceCalculator({ weights: adaptiveWeights })
  │       │       └─► calculator.calculateConfidence(incoming, latestSnapshot)
  │       │
  │       ├─► select bestMatch (highest score)
  │       │
  │       ├─► if bestMatch.score >= matchThreshold:
  │       │       deviceId = bestMatch.deviceId   (returning device)
  │       │   else:
```

```
│       │       deviceId = crypto.randomUUID()  (new device)
│       │
│       ├─► adapter.save({ deviceId, fingerprint: incoming, ... })
│       ├─► dedupCache.set(dedupKey, result, dedupWindowMs)
│       ├─► logger.info(...)
│       └─► metrics.incrementCounter(...)
│
└◄─ IdentifyResult { deviceId, confidence, isNewDevice, linkedUserId }
```

Extensibility & Customization

Custom Comparators

A `Comparator` is any function with the signature:

```
type Comparator = (value1: any, value2: any, path?: string) => number;
```

It must return a value in `[0, 1]` where `1.0` means identical and `0.0` means completely dissimilar. The optional `path` argument allows a single comparator to behave differently depending on which field it is invoked for.

Common comparator patterns include:

- **Levenshtein similarity** for string fields that change incrementally (e.g., user-agent minor version bumps).
- **Jaccard similarity** for set-valued fields (arrays where order does not matter).
- **Numeric proximity** for quantitative fields (e.g., `deviceMemory` treated as similar if within one doubling step).
- **Exact match** for high-entropy hash fields where any difference is significant.

Custom Storage Adapters

To integrate FP-Devicer with a database not covered by the built-in adapters, implement the `StorageAdapter` interface:

```
import { FPDataSet, StorageAdapter, StoredFingerprint } from "devicer.js";

const myAdapter: StorageAdapter = {
    async init() {/* open connection, create schema */},
    async save(snapshot: StoredFingerprint) {/* insert row */},
    async findCandidates(fp: FPDataSet, minScore: number, limit: number) {
        /* return StoredFingerprint[] of broadly-similar records */
        return [];
    },
    async getHistory(deviceId: string, limit: number) {
        return [];
    },
    async getAllFingerprints() {
        return [];
```

```
    },
    async linkToUser(deviceId: string, userId: string) {},
    async deleteOldSnapshots(olderThanMs: number) {},
    async close() {/* close connection */},
};
```

Custom Observability

Inject a `Logger` and/or `Metrics` implementation into `DeviceManager` to route telemetry to your platform (e.g., Winston, Pino, StatsD, Prometheus):

```
import { createInMemoryAdapter, DeviceManager } from "devicer.js";

const manager = new DeviceManager(createInMemoryAdapter(), {
    observability: {
        logger: {
            info: (msg, meta) => pinoLogger.info(meta, msg),
            warn: (msg, meta) => pinoLogger.warn(meta, msg),
            error: (msg, meta) => pinoLogger.error(meta, msg),
        },
        metrics: {
            incrementCounter: (name, value = 1) => statsd.increment(name, value),
            recordGauge: (name, value) => statsd.gauge(name, value),
            recordHistogram: (name, value) => statsd.histogram(name, value),
        },
    },
});
```

Performance & Accuracy

Benchmarking Methodology

FP-Devicer ships with a rigorous, self-contained benchmark suite under `src/benchmarks/`. All benchmarks run under the Vitest framework using its built-in `bench` harness, which applies statistical analysis across many iterations to produce stable, reproducible results. The suite is split into two concerns: **accuracy** (does the scorer make correct decisions?) and **performance** (how fast does it run?).

Synthetic Dataset Generation

All accuracy benchmarks operate on synthetic data produced by `data-generator.ts`. The generator is fully deterministic — it uses a seeded Linear Congruential Generator (LCG) as its PRNG so that results are reproducible across environments and CI runs. No external data sources or network calls are required.

For each simulated device a *base fingerprint* is constructed by sampling from realistic value pools:

- **Platform & OS** — 11 OS/architecture combinations covering Windows 10/11, macOS Intel/ARM, Ubuntu, Android 13/14, and iOS 16/17.
- **Browser** — 14 browser profiles (Chrome, Firefox, Safari, Edge, Opera across desktop and mobile) with versioned user-agent templates; version numbers are sampled from a realistic recency window.
- **Locale & timezone** — draws from a pool of 45 IANA timezone identifiers paired with culturally appropriate language lists.
- **Screen** — width × height × colour depth combinations sampled from 10 common display resolutions.
- **Hardware entropy** — `hardwareConcurrency` and `deviceMemory` sampled from common real-world values; UA-CH high-entropy hints populated accordingly.
- **Hash-based signals** — `canvas`, `webgl`, and `audio` fingerprints are generated by dedicated functions (`generateCanvasBlob`, `generateWebGLBlob`, `generateAudioBlob`) that produce multi-component strings mirroring the output of FP-Snatch's collection routines. Each function has a *stable* portion (seeded deterministically from the device seed, representing GPU/hardware baseline) and a *jitter* portion (varied per call, representing per-render floating-point noise). The raw strings are then passed through the same djb2 hash used by FP-Snatch.
- **Font list** — drawn from a pool of real system fonts; count and order vary per device.

Attractor Devices

Approximately 10–15% of simulated devices are designated **attractor devices**. These are generated by `createAttractorFingerprint` and represent common hardware/software profiles (e.g., default Windows 10 Chrome with a typical screen resolution) that produce fingerprints densely packed in feature space. Attractor devices are the hardest case for a fingerprint scorer because two completely different users may share nearly identical feature vectors. The `isAttractor` flag is propagated to every pair in the accuracy evaluation, and the `attr` column in the output table reports the False Accept Rate computed exclusively over impostor pairs where at least one party is an attractor device.

Mutation Model

Each device is assigned `sessionsPerDevice` fingerprint snapshots by applying the `mutate()` function at escalating intensity levels, cycling through `none → low → medium → high → low`. On 40% of non-`none` sessions, a second `low`-intensity pass is layered on top to ensure no two samples from the same device are ever bit-for-bit identical:

Level	Signals Modified
`none`	Clean baseline — no changes from the generated base
`low`	±1–2 px screen width rounding; canvas hash micro-jitter; 40% chance of font list reorder
`medium`	Chrome minor version bump; font addition or removal; 20% chance of timezone change; canvas and audio re-hash
`high`	Screen resolution change; Chrome major version jump (+2–6); DoNotTrack toggle; multiple font changes; canvas and WebGL hash drift from simulated driver update
`extreme`	Entirely new device profile generated from a fresh seed — treated as a different device for scoring purposes

The `extreme` level is excluded from the standard accuracy test corpus because it represents a hardware replacement, not a re-visit.

Pair Construction

The accuracy benchmark (`accuracy.bench.ts`) generates **5,750 scored pairs** from a corpus of 2,000 devices × 5 sessions each:

- **Genuine pairs** (`sameDevice = true`): For each iteration `i`, two distinct sessions from the same device are selected and passed to `calculateConfidence`. These represent legitimate re-visits that should score above the decision threshold.
- **Random impostor pairs** (`sameDevice = false`): A session from a separate device is paired with one from the target device. These represent different users that should score below the threshold.
- **Cross-browser impostor pairs** (`sameDevice = false`, deliberately harder): For 30% of iterations, a cross-browser pair is constructed by pairing a session from the target device with one sampled from an attractor-zone device. This tests resistance to the most difficult false-accept scenario.

All pair scores are computed synchronously at module load time before the Vitest bench harness starts timing, so DBSCAN analysis ($O(n^2)$) does not pollute the performance measurement.

Accuracy Metrics

Pair scores are evaluated across 21 thresholds from 0 to 100 in steps of 5. For each threshold the following metrics are computed:

Metric	Formula	Interpretation
Precision	$\mathrm{TP}\ /(\mathrm{TP}+\mathrm{FP})$	Of all pairs accepted, what fraction were genuine?
Recall	$\mathrm{TP}\ /(\mathrm{TP}+\mathrm{FN})$	Of all genuine pairs, what fraction were correctly accepted?
F1	$2 \cdot P \cdot R/(P+R)$	Harmonic mean of precision and recall
FAR	$\mathrm{FP}\ /(\mathrm{FP}+\mathrm{TN})$	Rate at which impostors are incorrectly accepted
FRR	$\mathrm{FN}\ /(\mathrm{TP}+\mathrm{FN})$	Rate at which genuine pairs are incorrectly rejected
EER	$\mid \mathrm{FAR}-\mathrm{FRR} \mid$	Proximity of FAR and FRR — minimum indicates the crossover point
attr	FAR over attractor-only impostor pairs	Hardened FAR for the most challenging impostor category

The threshold with the highest F1 score is selected as the recommended operating point. Results are persisted to `src/benchmarks/benchmark.out` after each run.

A complementary **DBSCAN analysis** (`eps = 0.05`, `minPts = 3`) is run over the full scored-pair distribution to characterise separability. DBSCAN groups pairs with similar scores into clusters; a large number of tight, distinct clusters indicates good bimodal separation between genuine and impostor score populations. The summary reports cluster count, noise points (pairs with anomalous scores not belonging to any cluster), total clustered points, and the largest cluster size.

Performance Benchmarks

`performance.bench.ts` measures end-to-end latency across the three deployment configurations most likely to be used in production:

Benchmark	What is timed
`calculateConfidence` (hybrid scorer)	Pure scoring only — structural comparison + TLSH blend, no I/O
`DeviceManager.identify` (In-Memory)	Full pipeline: dedup cache check, candidate scan, adaptive weighting, scoring, save
`DeviceManager.identify` (SQLite in-memory)	Same pipeline with SQLite `:memory:` as the storage backend
`DeviceManager.identify` (SQLite file-based)	Same pipeline with a real on-disk SQLite database, reflecting realistic conditions

Each adapter is seeded with 5 warmup `identify` calls before timing begins to ensure the dedup cache, query plan cache, and JIT compiler are in a steady state. The harness runs for 6 seconds per configuration (`time: 6000`) with a minimum of 50 iterations.

Benchmark Results

Results are from the most recent run against the current codebase on an 8-core x86_64 development machine. Timings are in milliseconds.

Accuracy (2,000 devices, 5,750 pairs)

Metric	Value
Best threshold	60
Precision at threshold 60	0.988
Recall at threshold 60	0.991
F1 at threshold 60	0.989
EER at threshold 60	0.000
FAR at threshold 60	0.010
FRR at threshold 60	0.009
Attractor impostor FAR (attr)	0.041
DBSCAN clusters	76
DBSCAN noise points	8 / 5750

The zero EER at threshold 60 is the Pareto-optimal operating point where FAR and FRR cross. The attractor FAR of 4.1% at this threshold represents the primary remaining risk: devices with common hardware profiles whose fingerprints cluster densely together can occasionally produce false accepts. This rate drops to zero above threshold 70, but at that point FRR rises to 12.6%, making threshold 60 the practical recommendation.

Performance

Configuration	Mean (ms)	p75 (ms)	p99 (ms)	Throughput (ops/sec)	RME
`calculateConfidence` (scorer only)	1.13	1.12	1.63	884	±1.74%
`DeviceManager` (In-Memory)	0.77	0.75	1.69	1,301	±0.52%
`DeviceManager` (SQLite in-memory)	0.78	0.78	1.54	1,274	±0.52%
`DeviceManager` (SQLite file-based)	0.76	0.75	1.32	1,321	±0.84%

The `DeviceManager` configurations outperform the raw scorer in mean latency because the dedup cache short-circuits repeated fingerprints before any scoring occurs. At steady-state traffic where a meaningful fraction of requests are repeat device visits, the effective throughput is substantially higher than the worst-case figures above.

SQLite file-based latency is comparable to in-memory at p75 and p99 due to SQLite's write-ahead logging and OS page cache effects; the higher tail at p999 (1.62 ms vs 2.29 ms for in-memory) reflects occasional fsync overhead.

To reproduce these results locally:

```
npm run bench
```

Results are written to `src/benchmarks/bench-results.json` (raw Vitest output) and `src/benchmarks/benchmark.out` (formatted accuracy tables).

Usage Examples

Method 1: Simple (Using Defaults)

```
import { calculateConfidence } from "devicer.js";

const score = calculateConfidence(fpData1, fpData2);
// score is an integer in [0, 100]
```

Method 2: Advanced (Custom Weights & Comparators)

```
import { createConfidenceCalculator, registerPlugin } from "devicer.js";

// Contribute a globally-scoped plugin before creating any calculators
registerPlugin("userAgent", {
    weight: 25,
    comparator: (a, b) =>
        levenshteinSimilarity(
            String(a ?? "").toLowerCase(),
            String(b ?? "").toLowerCase(),
        ),
});

const calculator = createConfidenceCalculator({
    weights: {
        platform: 20,
        fonts: 20,
        screen: 15,
    },
    tlshWeight: 0.2,
});

const score = calculator.calculateConfidence(fpData1, fpData2);
```

Method 3: Enterprise (DeviceManager with Express)

```
import express from "express";
import { createInMemoryAdapter, DeviceManager } from "devicer.js";

const manager = new DeviceManager(createInMemoryAdapter(), {
    matchThreshold: 60,
    candidateMinScore: 30,
});
await manager.adapter.init();

const app = express();
app.use(express.json());

app.post("/identify", async (req, res) => {
    const result = await manager.identify(req.body, {
        userId: (req as any).user?.id,
        ip: req.ip,
    });
    res.json(result);
    // → { deviceId, confidence, isNewDevice, linkedUserId }
});

app.listen(
    3000,
    () => console.log("FP-Devicer server ready at http://localhost:3000"),
);
```

Method 4: Deno / Oak (FP-Cicis Pattern)

```
import { Application, Router } from "oak";
import { DeviceManager } from "devicer";
import { createSqliteAdapter } from "./libs/sqlite.ts";

const adapter = createSqliteAdapter("./fp.db");
await adapter.init();

const manager = new DeviceManager(adapter, {
    matchThreshold: 60,
    candidateMinScore: 40,
});

const router = new Router();
router.post("/identify", async (ctx) => {
    const body = await ctx.request.body.json();
    const result = await manager.identify(body);
    ctx.response.body = result;
});
```

Installation & Integration

Installation

```
npm install devicer.js
```

FP-Snatch Integration

FP-Devicer is designed to pair with **FP-Snatch**, a companion open-source client-side JavaScript library that collects all fields of `FPUserDataSet` from a visitor's browser and POSTs them as JSON to a server endpoint.

```
<script src="./dist/bundle.js"></script>
<script defer>
    var agent = new window["snatch"]({
        url: "https://myserver.example/identify",
        method: "POST",
    });
    agent.send();
</script>
```

FP-Snatch collects, among other signals: user-agent, platform, screen dimensions, installed fonts, canvas fingerprint, WebGL fingerprint, audio fingerprint, plugin list, MIME types, hardware concurrency, device memory, timezone, language preferences, and UA-CH high-entropy values. It then sends the dataset as a JSON body matching the `FPUserDataSet` schema, ready for direct consumption by `DeviceManager.identify()` or `calculateConfidence()`.

A full working demonstration of the FP-Snatch → FP-Devicer pipeline is publicly hosted at cicis.info.

Conclusion & Future Work

FP-Devicer delivers production-grade, open-source device intelligence with unmatched extensibility and accuracy for its class. Its hybrid structural-plus-TLSH scoring, adaptive per-device stability weighting, multi-backend persistence options, and zero-dependency observability interfaces set it apart from simpler fingerprint hash libraries.

The library is suitable for a wide range of use cases: account security and fraud detection, analytics de-duplication, personalisation without cookies, and research into browser fingerprinting techniques.

Community contributions are welcome. Possible future directions include:

- Official npm publishing and semantic versioning automation
- Additional storage adapters (DynamoDB, MongoDB, Cloudflare KV)
- Machine-learning-assisted comparator parameter tuning
- Browser extension support for richer signal collection
- Differential privacy mechanisms for privacy-preserving fingerprinting

License

See `license.txt` in the repository root. FP-Devicer is released under a permissive open-source license.